Survey for Hardware Aware Neural Architecture Search for SNNs

Luo Ruijue
Supervisors: Hella Toto Kiesa, Foivos Paraskevas
Institut für Technik der Informationsverarbeitung (ITIV)
22.08.2025

Neural Architecture Search

- Neural Architecture Search: Automated design of neural networks

- Traditional Neural Architecture Search
  - Focus: Accuracy
  - Assumes powerful hardware
  - Large/inefficient models possible

- Hardware Aware Neural Architecture Search
  - Accuracy + Hardware efficiency
  - Metrics: latency, energy, memory, area

Spiking Neural Networks

- Spiking Neural Networks (SNNs)[1]
  - Biological inspired
  - Encode information in spike timing
  - Energy-efficient
  - Neuron model: LIF (Leaky Integrate-and-Fire)

- Neuromorphic Hardware
  - IBM TrueNorth, Intel Loihi

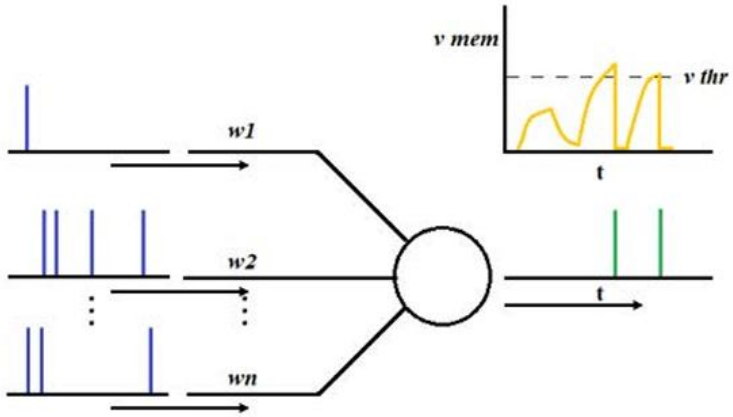

Fig. 1. Working principle of a spiking neuron[1]

Problem Statement

- Why SNNs?[1]
  - Event-driven, low energy
  - Neuromorphic hardware friendly
- Challenges[1]
  - Non-differentiable spikes
  - Hardware constraints not considered in traditional NAS
- Problem: How to search for architectures optimized for both task and hardware?

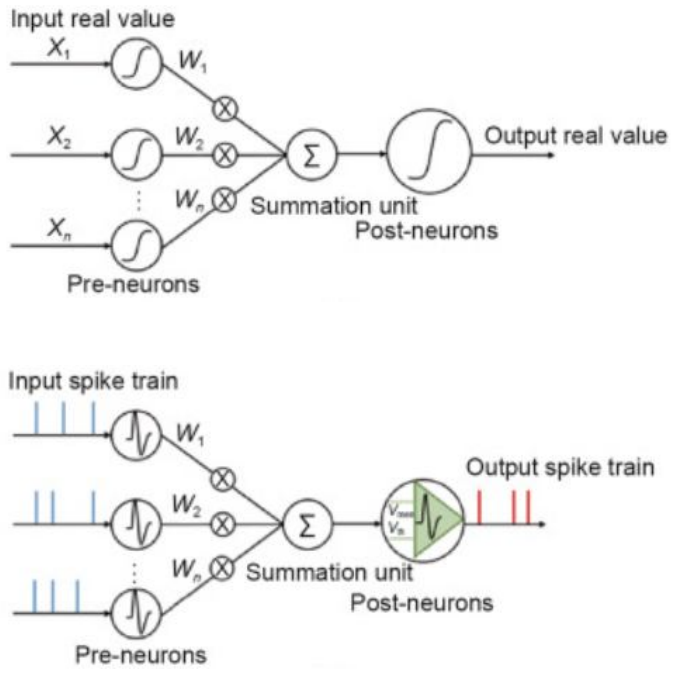

Fig. 2. Comparison between ANN and SNN[2]

Evolutionary Algorithms

- Why EAs for SNNs?[3]
  - Gradient-free, suitable for SNNs
- How do they work?[3]
  - Population-based search: start with random architectures
  - Iterative improvement: evaluate, select, crossover, mutate
  - Objective: optimize fitness (accuracy + energy + latency)
- Examples: AutoSNN[4]

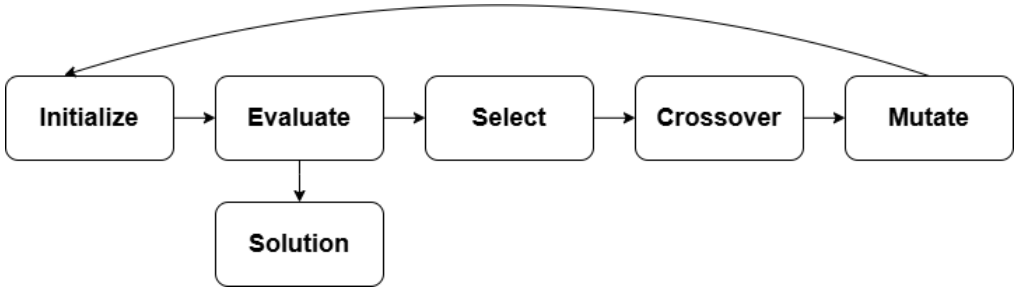

Fig. 3. General workflow of an Evolutionary Algorithm (EA)

Differentiable NAS

- Why DNAS for SNNs?
  - Gradient descent → faster than EA
  - Scales to large search spaces

- How does it work?[6]
  - Candidate operations (ops) on each edge
  - Continuous relaxation via softmax weighting
  - Optimization of architecture parameters with surrogate gradients
  - Derivation of final discrete architecture

- Example: DARTS[6]

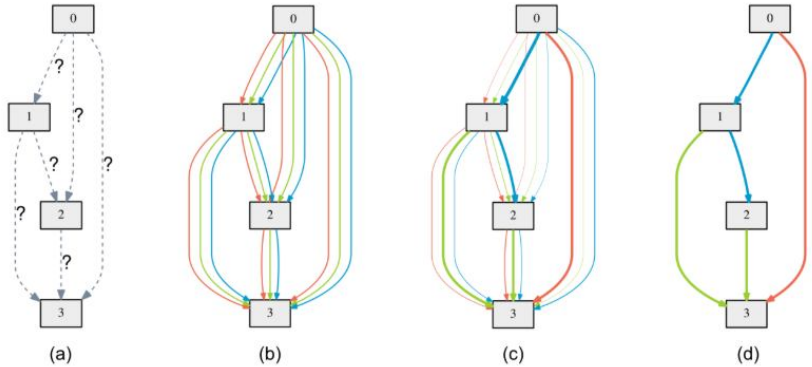

Fig. 4. Workflow of Differentiable NAS (DNAS)

Multi - Objective Search

- Why multi-objective search for SNNs?
  - Real hardware needs accuracy and efficiency (latency, energy, memory)
- How does it work?
  - Pareto frontier optimization
    - No single best model, but a set of trade-off solutions
  - Accuracy + Latency + Energy simultaneously
- Example: HASNAS[5], MONAS-ESNN[7]

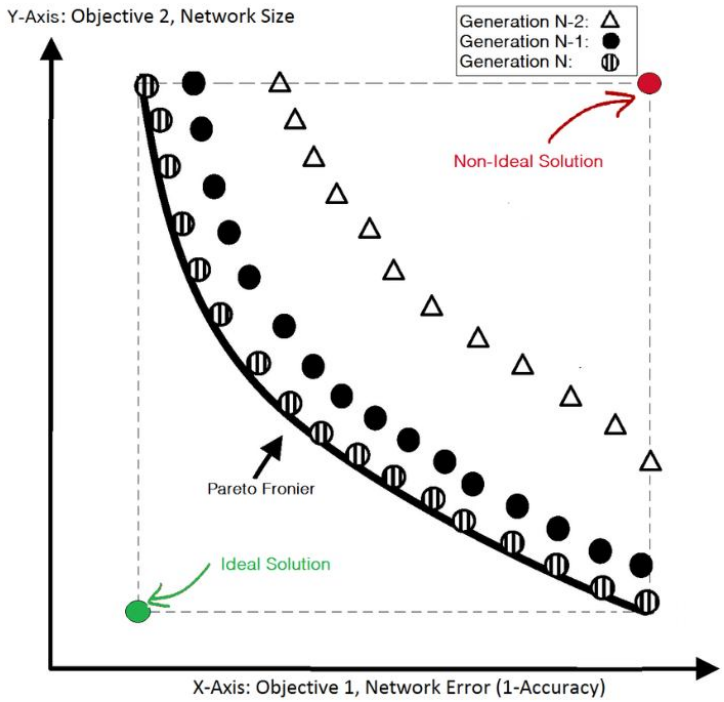

Fig. 5. Pareto frontier illustration[8]

Zero Cost Proxies

- Why zero-cost proxies for SNNs?
  - Full training is too expensive for large search spaces[9]
  - Traditional proxies still require computation and do not generalize well
    - Partial Training
    - Floating Point Operations (FLOPs)
    - Parameter Count

Zero Cost Proxies

- How does it work?
  - Replace full training with cheap proxies
    - Parameter count (structural complexity)[10]
    - Spike activity / firing rate (energy proxy)[11]
- What does it provide?
  - Fast filtering of bad candidates

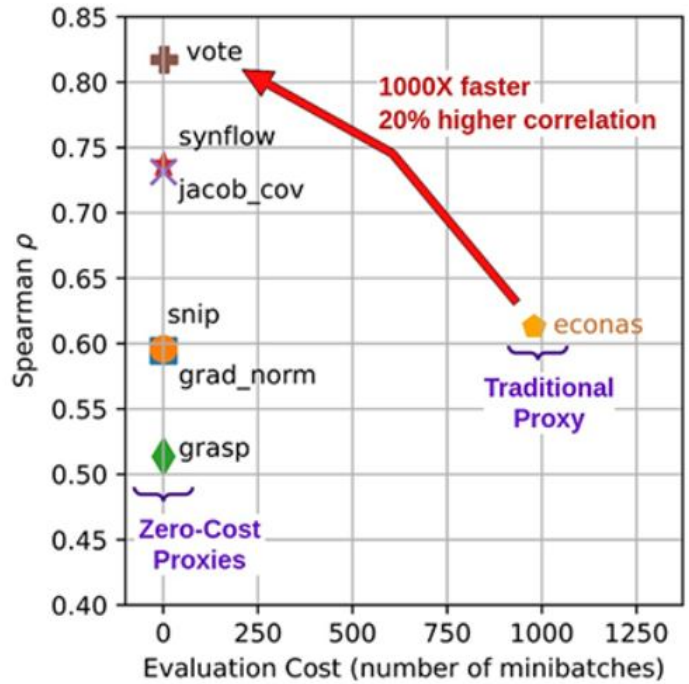

Fig. 6. Comparison of Zero-Cost and Traditional Proxies[12]

Comparison of HW-NAS Techniques for SNNs

Framework	Search Method	Proxy Type	Multi-Objective (MOS)	Key Limitation
AutoSNN[4]	EA	energy proxy	Accuracy & Spike count	No real hardware modeling
DARTS[6]	DNAS	Validation set loss	No, accuracy only	No real hardware modeling
HASNAS[5]	Cell-based + Operator analysis	Cost models from CIM Literatures	Accuracy & HW constraints	Proxy modeling, no real chip
MONAS-ESNN[7]	EA	Zero-Proxy + Spike count	Accuracy & Spike count	Proxy ≠ real accuracy/energy

Discussion

- Trade-offs
  - Evolutionary Algorithms
    - Strong global search capability, good at finding diverse solutions
    - Drawback: Computationally expensive, slow convergence
  - Differentiable NAS
    - Very fast convergence by using gradient-based optimization
    - Drawback: Less flexible, struggles with discrete/spike-based search spaces
  - Multi-Objective Search
    - Balances multiple hardware and accuracy objectives simultaneously
    - Drawback: Complex optimization process, higher design overhead

Conclusion

- EA is preferable for small-scale SNNs or when accuracy and diversity matter most
- DNAS is suitable for fast prototyping under limited search space
- MOS works best for real deployment where latency and energy efficiency must be balanced

The choice of method depends on application requirements (accuracy, speed, hardware constraints).

Literature

[1] Y. Wu, L. Deng, G. Li, J. Zhu, and L. Shi, "Spiking Neural Networks: A Survey," IEEE Transactions on Neural Networks and Learning Systems, vol. 34, no. 3, pp. 635–655, Mar. 2023, doi: 10.1109/TNNLS.2022.3149445.

[2] Y.-H. Wang, T.-C. Gong, Y.-X. Ding, J. Wu, C. Xu, J.-J. Yang, and R.-S. Han, “Redox memristors with volatile threshold switching behavior for neuromorphic computing,” Nature Communications, vol. 13, no. 1, pp. 1–11, 2022, doi: 10.1038/s41467-022-33412-6.

[3] N. G. Pavlidis, D. K. Tasoulis, V. P. Plagianakos, and M. N. Vrahatis, “Spiking neural network training using evolutionary algorithms,” in Proc. 2005 IEEE International Joint Conference on Neural Networks, vol. 4, pp. 2190–2194, 2005, doi: 10.1109/IJCNN.2005.1556245.

Literature

[4] S. Rathi, S. Panda, P. Panda, and K. Roy, “AutoSNN: Towards energy-efficient spiking neural networks,” IEEE Transactions on Neural Networks and Learning Systems, vol. 33, no. 4, pp. 1446–1457, Apr. 2022, doi:10.1109/TNNLS.2020.3048340.

[5] Putra et al., ”HASNAS: A Hardware-Aware Spiking Neural Architecture Search Framework for Neuromorphic Compute-in-Memory Systems,” IEEE Trans. Comput.-Aided Design Integr. Circuits Syst., vol. 42, no.5, pp. 1521–1534, May 2023.

[6] H. Liu, K. Simonyan, and Y. Yang, "DARTS: Differentiable architecture search," in Proc. Int. Conf. Learn. Represent. (ICLR), 2019.

Literature

[7] E. S. Ghasemi and S. K. Lai-Yuen, “MONAS-ESNN: Multi-Objective Neural Architecture Search for Efficient Spiking Neural Networks,” in Proc. IEEE/CVF Winter Conf. on Applications of Computer Vision (WACV), 2025, pp. 2963–2972. doi: 10.1109/WACV61041.2025.00891

[8] M. Loni, N. D. Lane, and M. Bahrami, "ADONN: Adaptive design of optimized deep neural networks for embedded systems," Pattern Recognition, vol. 124, p. 108499, 2022.

[9]M. S. Abdelfattah, C. Zhang, D. Ren, A. Jha, Y. He and L. Shen, “Zero-Cost Proxies for Lightweight NAS,” in Proc. Int. Conf. Learn. Representations (ICLR), 2021.

Literature

[10]B. Zoph and Q. V. Le,“Neural architecture search with reinforcement learning,” in Proc. Int. Conf. Learn. Representations (ICLR), 2017.

[11]L. Deng, Y. Wu, Z. Hu, L. Liang, G. Ding and Y. Li, “Optimal Conversion of Conventional Artificial Neural Networks to Spiking Neural Networks,” in Proc. Int. Conf. Learn. Representations (ICLR), 2021

[12] Samsung Research Blog, “Zero-Cost Neural Architecture Search,” Samsung Research, 2021. [Online]. Available: https://research.samsung.com/blog/Zero-Cost-Neural-Architecture-Search

BACKUP

AutoSNN

- AutoSNN: Towards energy-efficient spiking neural networks
- Propose NAS framework for SNNs
- Two-level search space
  - Macro-level: overall backbone (depth, layer types, pooling)
  - Micro-level: local spiking blocks (kernel size, neuron type, residuals)
- Evolutionary Algorithm (EA) for exploration + Multi-objective optimization (MOS)

DARTS

- DARTS: Differentiable architecture search

- Cell-based search
  - Network = stack of Normal Cell + Reduction Cell
- Continuous relaxation
- Bi-level optimization
  - Inner loop: update weights on train set
  - Outer loop: update architecture on val set
- Final architecture
  - Discretize by picking max-weight operator per edge
  - Retrain from scratch

HASNAS

- HASNAS: Hardware-Aware Spiking Neural Architecture Search for Neuromorphic Compute-in-Memory Systems

- HASNAS extends to hardware-level Multi-objective optimization
  - Prior NAS for SNNs only used spike count as energy proxy

- Operation-level analysis → prune inefficient ops (e.g., large kernels, FC layers)
- Cell-based architecture → Cell-A / Cell-B as Macro building blocks

- Multi-objective optimization (Memory / Area / Latency / Energy)

HASNAS

- Multi-objective optimization constraint:
  - Memory ≤ 4M
  - Area ≤ 100 mm²
  - Latency ≤ 400 ms
  - Energy ≤ 120 μJ

- HASNAS achieves up to 11× faster search compared to the full search space baseline, by leveraging operator-level pruning and cell-based design.

MONAS-ESNN

- MONAS-ESNN: Multi-Objective NAS for Efficient Spiking Neural Networks

- Zero-Proxy Evaluation
  - Proposed ASAHD (Adjusted Sparsity-Aware Hamming Distance)
  - Measures spike separability without training
- Multi-Objective Optimization
  - search with evolutionary algorithm
  - Maximize ASAHD (proxy accuracy) + Minimize spike count (proxy energy)

Links

AutoSNNs: https://github.com/nabk89/AutoSNN

DARTS: https://github.com/quark0/darts

HASNAS: https://www.emergentmind.com/papers/2407.00641

MONAS-ESNN: https://ieeexplore.ieee.org/abstract/document/10944107